36

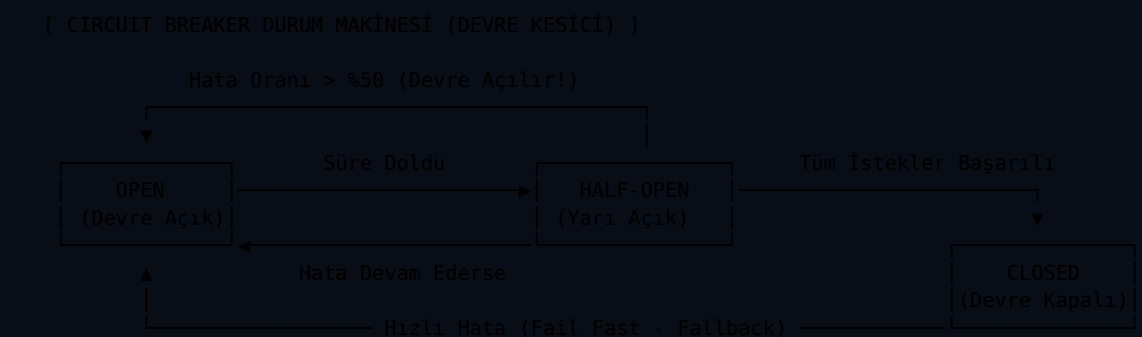
BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Hata İzolasyonu ve Dayanıklılık: Circuit Breaker, Bulkhead ve Jitter'lı Yeniden Deneme

Fault Tolerance & Resiliency: Circuit Breaker, Bulkhead & Jittered Retry

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1) Circuit Breaker (Evdeki Sigorta): Evde prizden kıvılcım çıkınca tüm binanın yanmasını beklemezsiniz; sigorta 'ÇAT' diye atar (Open). Sorunlu odaya giden elektrik kesilir ama evin geri kalanı yaşamaya devam eder. 2) Bulkhead (Gemideki Su Geçirmez Bölmeler): Titanik gibi devasa bir geminin gövdesi 10 ayrı su geçirmez odaya bölünmüştür. Bir oda delinip su alsa bile su diğer odalara geçmez ve gemi batmaz. 3) Jitter'lı Yeniden Deneme: 100 kişinin aynı kapıyı aynı anda tekmelemesi yerine, herkesin rastgele 1 ile 5 saniye bekleyip kapıyı çalmasıdır!

🇬🇧 ENGLISH (ELI5)

1) Circuit Breaker (Electrical Fuse): When a socket sparks, you don't let the entire house burn down; the fuse trips 'SNAP!' (Open). Electricity isolates that room while the rest of the house stays lit. 2) Bulkhead (Ship Hull Compartments): A submarine hull divides into isolated watertight chambers. If chamber 3 floods, steel doors seal it off; the submarine stays buoyant. 3) Exponential Backoff with Jitter: Instead of 1,000 clients furiously hammering an overwhelmed server every exact 1.0 second (Retry Storm), each client waits an exponentially growing duration plus random jitter (e.g. 1.2s, 2.7s, 5.1s) to disperse the thundering herd!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık dayanıklılığın 3 altın deseni: 1) Circuit Breaker (Resilience4j / Envoy): Kapalı (Normal), Açık (Hata oranı %50'yi aşarsa istekleri uzak servise göndermeden doğrudan Fallback döner - Fail-Fast), Yarı-Açık (Süre dolunca deneme amaçlı birkaç istek geçirir). 2) Bulkhead: Thread havuzlarını ve bağlantı havuzlarını birbirinden izole eder (ör. Ödeme servisi yavaşlasa bile Arama servisinin thread havuzunu tüketip tüm pod'u kilitleyemez). 3) Exponential Backoff with Full Jitter: Uzak servis çöktüğünde yeniden deneme süresini üstel olarak artırmak yetersizdir; tüm istemciler aynı anda denerse 'Retry Storm' oluşur. Formül: Sleep = rand(0, min(Cap, Base × 2^attempt)).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The trifecta of distributed stability: 1) Circuit Breaker: States: Closed (normal execution), Open (failure threshold exceeded, failing-fast immediately with graceful degraded fallback responses), Half-Open (canary testing backend recovery). 2) Bulkhead Pattern: Partitions thread pools, CPU allocations, and database connection pools across domain features. A hung Recommendation engine cannot exhaust the thread pool dedicated to Payment processing. 3) Exponential Backoff with Full Jitter: Simple exponential backoff synchronizes retry spikes into destructive waves (Retry Storms). Amazon AWS proved that injecting randomized Full Jitter (t = random(0, min(M, B × 2^i))) flattens concurrency peaks into smooth traffic profiles.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Exponential Backoff with Full Jitter Implementasyonu
import java.util.concurrent.ThreadLocalRandom;

public class ResilientRetry {
    public static long calculateFullJitterSleep(int attempt, long baseMillis, long capMillis) {
        long exponential = Math.min(capMillis, baseMillis * (1L << attempt));
        return ThreadLocalRandom.current().nextLong(0, exponential + 1);
    }
}

void main() {
    long base = 100; // 100ms
    long cap = 5000; // 5000ms tavan

    IO.println("--- Jitter'lı Yeniden Deneme Bekleme Süreleri ---");
    for (int attempt = 0; attempt < 5; attempt++) {
        long sleep = ResilientRetry.calculateFullJitterSleep(attempt, base, cap);
        IO.println("Deneme #" + (attempt + 1) + " -> Uyku: " + sleep + " ms");
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bağlantı hatası olursa hemen while(true) döngüsüyle tekrar denerim' demeyin! Kontrolsüz yeniden denemeler çökmekte olan bir sunucuyu 'Retry Storm' ile tamamen öldürür. Her zaman Circuit Breaker, Exponential Backoff ve Jitter birlikte kullanılmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch immediate tight-loop retries. Unconstrained retries create catastrophic Retry Storms that obliterate recovering downstream dependencies. Always combine Circuit Breakers with Exponential Backoff and Full Jitter.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Circuit Breaker yavaşlayan servislere giden trafiği keserek hızlı hata (Fail-Fast) üretir.

• Bulkhead deseni kaynak havuzlarını izole ederek tek bir servisin tüm pod'u kilitlemesini önler.

• Full Jitter, yeniden deneme fırtınalarını (Retry Storms) dağıtarak sistemi yumuşakça kurtarır.

🎯 Key Takeaways for the Interview

• Circuit Breakers fail fast during downstream outages, preserving upstream resource capacity.

• The Bulkhead pattern partitions thread pools, isolating failures to single sub-systems.

• Full Jitter stochastically disperses retries, dissipating destructive thundering herds.